

美团前端架构师面试-数据权限方向

一面（技术基础）

1. 请简单介绍一下 CSS 的盒模型，以及 box-sizing 属性的作用。

难度：★

考点：CSS 布局基础

答案要点：

盒模型是浏览器渲染引擎对 HTML 元素进行布局时所遵循的规则，它将所有元素看作一个矩形的盒子。

- 组成部分：由 content（内容）、padding（内边距）、border（边框）和 margin（外边距）组成。
- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- 怪异盒模型：width 包含了 content、padding 和 border。
- box-sizing：通过 content-box（标准）和 border-box（怪异）进行切换，实际开发中常用 border-box 以便控制尺寸。

常见坑：

- 混淆 margin 纵向重叠（Margin Collapsing）的情况。
- 忽略行内元素（inline）对某些盒模型属性（如 width/height）的不生效问题。

追问：

- 如何触发 BFC 来解决 margin 重叠？
- 什么是替换元素，它们的盒模型有什么特殊之处？

2. 谈谈你对 JavaScript 原型和原型链的理解。

难度：★★

考点：JS 核心原理

答案要点：

原型是 JS 实现继承的基础，每个对象都有一个指向其构造函数原型对象的内部链接。

- 构造函数与原型：每个函数都有 prototype 属性，指向其实例的原型对象。
- 隐式原型：每个实例对象都有 **proto** 属性，指向构造函数的 prototype。
- 原型链：当访问一个对象的属性时，如果自身没有，会沿着 **proto** 向上查找，直到 null。
- 终点：原型链的顶端是 Object.prototype.**proto**，即 null。

常见坑：

- 误认为所有对象都有 prototype 属性（只有函数有）。
- 忽略了 constructor 属性的指向问题。

追问：

- 如何判断一个属性是存在于对象本身还是原型链上？
- Object.create(null) 创建的对象有原型吗？

3. 请解释一下闭包是什么，以及它在实际开发中的应用场景。

难度：★★

考点：作用域与内存管理

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑在一起。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 核心作用：延伸了变量的作用范围，使其在外部函数执行完后不被销毁。
- 应用场景：封装私有变量、柯里化（Currying）、防抖节流、模块化开发。
- 内存影响：由于引用存在，可能导致变量无法被垃圾回收，需注意内存泄漏。

常见坑：

- 在循环中使用 var 定义变量导致闭包取值错误。
- 过度使用闭包导致性能下降。

追问：

- 闭包会导致内存泄漏吗？如何手动释放？
- 现代浏览器对闭包的优化有哪些？

4. 浏览器中的事件循环（Event Loop）机制是怎样的？

难度：★★

考点：异步执行机制

答案要点：

事件循环是 JS 处理异步任务的核心机制，它通过任务队列协调主线程的执行。

- 任务分类：分为同步任务、微任务（Microtask）和宏任务（Macrotask）。
- 执行顺序：先执行同步代码，然后清空当前的微任务队列，最后取出一个宏任务执行。
- 微任务：Promise.then、MutationObserver、process.nextTick (Node)。
- 宏任务：setTimeout、setInterval、I/O、UI Rendering。

常见坑：

- 误以为 Promise 构造函数里的代码是异步的。
- 混淆 await 后面的代码执行时机。

追问：

- 为什么 Promise 的回调比 setTimeout 先执行？
- 浏览器渲染（Render）发生在事件循环的哪个阶段？

5. 请手写一个基础的防抖（Debounce）函数。

难度：★★

考点：手写代码能力

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 核心逻辑：利用闭包保存 timer 变量。
- 清除定时器：每次触发时先 clearTimeout。
- 绑定上下文：确保 this 指向正确，参数正常传递。

代码块

```
1 function debounce(fn, delay) {  
2   let timer = null;  
3   return function(...args) {  
4     if (timer) clearTimeout(timer);  
5     timer = setTimeout(() => {  
6       fn.apply(this, args);  
7     }, delay);  
8   };  
9 }
```

常见坑：

- 忘记处理 this 指向。
- 忘记传递 arguments 参数。

追问：

- 如果要求第一次触发立即执行，该如何修改？
- 防抖和节流的区别是什么？

6. 说说 Promise 的几种状态，以及 Promise.all 和 Promise.race 的区别。

难度：★

考点：异步编程 API

答案要点：

Promise 是异步编程的一种解决方案，它有三种状态且状态一旦改变不可逆。

- 三种状态：Pending（进行中）、Fulfilled（已成功）、Rejected（已失败）。
- Promise.all：接收数组，所有 Promise 都成功才返回成功数组，有一个失败则立即返回失败。
- Promise.race：接收数组，返回最先改变状态（无论成功失败）的那个 Promise 的结果。
- 链式调用：then 方法返回的是一个新的 Promise 实例。

常见坑：

- 忘记在 Promise 中处理 catch 导致未捕获异常。
- 误认为 Promise.all 里的任务是串行执行的。

追问：

- 如果 Promise.all 中某个任务失败了，但不希望整个 all 失败，该怎么处理？
- Promise.allSettled 的作用是什么？

7. HTTP 状态码 204、301、302、304 分别代表什么含义？

难度：★

考点：网络协议基础

答案要点：

状态码是服务器对请求处理结果的简短反馈。

- 204 No Content：请求成功，但响应报文不包含实体主体。
- 301 Moved Permanently：永久重定向，搜索引擎会更新索引。
- 302 Found：临时重定向，搜索引擎不会更新索引。
- 304 Not Modified：协商缓存命中，表示资源未改变，直接使用本地缓存。

常见坑：

- 混淆 301 和 302 对 SEO 的影响。
- 不清楚 304 触发的前提条件（If-Modified-Since 等）。

追问：

- 307 和 302 有什么区别？
- 常见的 400 系列错误有哪些？

8. 如何实现一个元素的水平垂直居中？

难度：★

考点：CSS 布局实操

答案要点：

实现居中的方式有很多，取决于具体的场景和兼容性要求。

- Flex 布局：display: flex; justify-content: center; align-items: center;
- Grid 布局：display: grid; place-items: center;
- 绝对定位 + Transform：top: 50%; left: 50%; transform: translate(-50%, -50%);
- 绝对定位 + 负 Margin：已知宽高时使用。

常见坑：

- 绝对定位时忘记给父元素设置 position: relative。
- transform 导致文字模糊或层级问题。

追问：

- 如果是行内块元素（inline-block），如何实现垂直居中？
- 谈谈 vertical-align 的生效条件。

9. 数组有哪些常用的方法？请区分哪些会改变原数组。

难度：★

考点：JS 基础 API

答案要点：

JS 提供了丰富的数组操作方法，根据是否修改原对象可分为变异方法和非变异方法。

- 改变原数组：push, pop, shift, unshift, splice, sort, reverse。
- 不改变原数组：map, filter, reduce, slice, concat, forEach。
- 遍历方法：forEach 无法中断，map 返回新数组，reduce 用于累加。

常见坑：

- 误以为 map 会改变原数组。
- splice 和 slice 参数含义记混。

追问：

- 如何清空一个数组？
- 怎么判断一个变量是否为数组？

10. 什么是跨域？常见的解决跨域的方案有哪些？

难度：★★

考点：浏览器安全与通信

答案要点：

跨域是由浏览器的同源策略引起的，即协议、域名、端口三者有任一不同。

- CORS：后端设置 Access-Control-Allow-Origin 等响应头，是目前的主流方案。

- JSONP：利用 script 标签不受同源策略限制的特性，仅支持 GET 请求。
- 反向代理：通过 Nginx 或 Webpack Dev Server 将请求转发到同源接口。
- postMessage：用于跨窗口、跨 iframe 的通信。

常见坑：

- 认为跨域是请求发不出去（实际上请求发出了，响应被浏览器拦截）。
- 忽略了简单请求和预检请求（Preflight）的区别。

追问：

- CORS 预检请求（OPTIONS）在什么情况下会触发？
- Cookie 在跨域请求中如何携带？

11. 请描述一下从输入 URL 到页面加载完成的过程。

难度：★★

考点：综合基础知识

答案要点：

这是一个涉及网络、解析、渲染的复杂过程。

- 网络阶段：DNS 解析、TCP 三次握手、发送 HTTP 请求、接收响应。
- 解析阶段：解析 HTML 生成 DOM 树，解析 CSS 生成 CSSOM 树。
- 渲染阶段：合并生成 Render Tree，布局（Layout），绘制（Paint），合成（Composite）。
- 脚本执行：遇到 script 标签会阻塞解析（除非有 defer/async）。

常见坑：

- 漏掉 DNS 缓存或 TCP 握手环节。
- 认为 DOM 树构建完成后才开始解析 CSS。

追问：

- 为什么建议将 CSS 放在头部，JS 放在底部？
- 重排（Reflow）和重绘（Repaint）的区别是什么？

12. 简单实现一个浅拷贝和深拷贝。

难度：★★

考点：对象操作

答案要点：

浅拷贝只复制对象的第一层属性，深拷贝则递归复制所有层级。

- 浅拷贝实现：Object.assign({}, obj) 或扩展运算符 {...obj}。
- 深拷贝简单实现：JSON.parse(JSON.stringify(obj))，但无法处理函数、正则、循环引用。

- 递归实现：判断类型，递归遍历属性，处理循环引用（使用 Map）。

代码块

```
1  function deepClone(obj, hash = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') return obj;
3    if (hash.has(obj)) return hash.get(obj);
4    let res = Array.isArray(obj) ? [] : {};
5    hash.set(obj, res);
6    for (let key in obj) {
7      if (obj.hasOwnProperty(key)) {
8        res[key] = deepClone(obj[key], hash);
9      }
10   }
11   return res;
12 }
```

常见坑：

- 递归深拷贝时没有处理循环引用导致栈溢出。
- 忽略了 Date、RegExp 等特殊对象的处理。

追问：

- structuredClone API 了解吗？
- 为什么不推荐在所有场景都用 JSON.stringify 做深拷贝？

二面（深入原理与项目）

1. 谈谈 React 的 Fiber 架构，它解决了什么问题？

难度：★★★

考点：框架底层原理

答案要点：

Fiber 是 React 16 引入的一种新的协调引擎，旨在提高复杂应用的响应速度。

- 解决痛点：旧版 Stack Reconciler 递归执行，任务过重时会阻塞主线程，导致掉帧。
- 核心机制：将渲染工作拆分为小单元（Fiber），支持任务的暂停、恢复和优先级排序。
- 双缓存技术：在内存中构建 workInProgress tree，完成后一次性提交到真实 DOM。
- 调度过程：利用 requestIdleCallback（模拟实现）在浏览器空闲时执行任务。

常见坑：

- 误认为 Fiber 提高了渲染的绝对速度（其实是提高了交互的流畅度）。

- 混淆了 Render 阶段（可中断）和 Commit 阶段（不可中断）。

追问：

- Fiber 节点上的 key 有什么作用？
- 为什么 React Hooks 必须在顶层调用？

2. 你们项目里是如何进行性能优化的？请列举具体的手段。

难度：★★★

考点：工程化实践

答案要点：

性能优化需要从网络加载、资源体积、渲染效率三个维度展开。

- 网络优化：使用 CDN、开启 Gzip/Brotli、HTTP2 协议、预加载（preload/prefetch）。
- 体积优化：Webpack 代码分割（Code Splitting）、Tree Shaking、图片压缩、按需加载组件库。
- 渲染优化：减少重排重绘、虚拟列表、防抖节流、React.memo/useMemo、骨架屏。
- 指标监控：关注 LCP、FID、CLS 等核心 Web 指标。

常见坑：

- 盲目使用 memo 导致额外的内存开销。
- 优化了非瓶颈环节，导致投入产出比低。

追问：

- 如何定位页面卡顿的原因？
- 什么是“长任务（Long Task）”，如何拆解它？

3. Vue 2 和 Vue 3 的响应式原理有什么区别？

难度：★★

考点：框架对比

答案要点：

Vue 3 改进了响应式系统，解决了 Vue 2 存在的局限性并提升了性能。

- Vue 2：基于 Object.defineProperty，通过 getter/setter 拦截属性。
- Vue 3：基于 ES6 Proxy，拦截整个对象的操作。
- 优势对比：Proxy 可以监听数组下标变化、动态添加/删除属性，且性能更好（懒代理）。
- 局限性：Vue 2 无法检测对象属性的添加或删除，需要使用 \$set。

常见坑：

- 认为 Proxy 会对嵌套对象进行全量递归（实际上是访问到哪层才代理哪层）。
- 忽略了 Proxy 对旧版浏览器的兼容性问题。

追问：

- Vue 3 的 Composition API 解决了什么问题？
- ref 和 reactive 的区别是什么？

4. 请描述一下 Webpack 的构建流程。

难度：★★★

考点：构建工具原理

答案要点：

Webpack 是一个串行的构建过程，通过插件机制（Tapable）贯穿整个生命周期。

- 初始化：读取配置，加载插件，实例化 Compiler。
- 编译阶段：从 Entry 出发，调用 Loader 翻译模块，递归构建依赖图（Dependency Graph）。
- 生成阶段：将编译后的模块组合成 Chunk，转换成 Bundle 文件。
- 输出阶段：根据配置的 output 将文件写入文件系统。

常见坑：

- 混淆 Loader（转换器）和 Plugin（扩展器）的作用。
- 不清楚热更新（HMR）的基本原理。

追问：

- 如何编写一个自定义的 Loader？
- Webpack 5 相比 Webpack 4 有哪些重大改进？

5. 什么是虚拟 DOM？它真的比直接操作真实 DOM 快吗？

难度：★★

考点：渲染性能

答案要点：

虚拟 DOM 是对真实 DOM 的一层抽象，本质是 JS 对象。

- 核心价值：通过 Diff 算法计算最小变更，减少不必要的 DOM 操作；提供跨平台能力。
- 性能误区：直接操作 DOM 永远是最快的，虚拟 DOM 的优势在于在保证性能下限的同时，极大提高了开发效率。
- 批量更新：将多次状态变更合并为一次 DOM 修改。

常见坑：

- 认为虚拟 DOM 无论什么场景都比原生快。
- 忽略了 Diff 算法本身的计算开销。

追问：

- React 和 Vue 的 Diff 算法有什么异同？
- 为什么 Diff 算法通常是同层比较？

6. 谈谈前端安全，如何防范 XSS 和 CSRF 攻击？

难度：★★

考点：Web 安全

答案要点：

安全是前端开发不可忽视的一环，主要针对脚本注入和身份伪造。

- XSS（跨站脚本攻击）：对用户输入进行转义、使用 CSP 策略、设置 HttpOnly 属性。
- CSRF（跨站请求伪造）：使用 Token 验证、验证 Referer/Origin、设置 Cookie 的 SameSite 属性。
- 区别：XSS 是在页面内执行恶意脚本，CSRF 是借用用户的登录状态发起伪造请求。

常见坑：

- 认为设置了 HTTPS 就能防范 XSS/CSRF。
- 混淆了存储型 XSS 和反射型 XSS。

追问：

- 什么是点击劫持（Clickjacking）？如何防御？
- 为什么 JWT 相对 Session 更有优势？

7. 你们项目中是如何处理前端异常监控的？

难度：★★★

考点：工程化与稳定性

答案要点：

异常监控需要覆盖运行时错误、资源加载失败和接口异常。

- 捕获手段：window.onerror 捕获同步错误，unhandledrejection 捕获 Promise 错误。
- 框架层面：React 的 Error Boundary，Vue 的 errorHandler。
- 上报策略：采样上报、错误去重、利用 navigator.sendBeacon 保证页面卸载时上报成功。
- 辅助信息：上报 UserAgent、路由信息、操作日志、SourceMap 还原堆栈。

常见坑：

- 忽略了跨域脚本错误的捕获（Script error）。
- 上报数据量过大导致服务器压力。

追问：

- 如何实现白屏监控？

- 线上代码混淆后，如何根据报错定位到源代码？

8. 谈谈你对微前端的理解，以及常见的实现方案。

难度：★★★

考点：架构设计

答案要点：

微前端是将大型应用拆分为多个独立交付的微应用的架构风格。

- 核心价值：技术栈无关、独立开发部署、增量升级。
- 实现方案：iframe（隔离性最好但体验差）、single-spa（路由分发）、qiankun（基于 single-spa 的封装）、Module Federation（Webpack 5 模块联邦）。
- 关键问题：样式隔离（Shadow DOM/CSS Scoping）、JS 沙箱（Proxy 代理）、应用间通信。

常见坑：

- 忽略了公共依赖的提取和共享。
- 微应用之间的全局变量污染问题。

追问：

- qiankun 是如何实现 JS 沙箱的？
- 微前端架构下，如何处理全局状态管理？

9. 什么是虚拟列表（Virtual List）？它的实现原理是什么？

难度：★★

考点：长列表优化

答案要点：

虚拟列表是一种只渲染可视区域内数据，以提升长列表性能的技术。

- 实现原理：监听滚动事件，根据滚动距离计算当前应显示的起始索引和结束索引。
- 关键计算：容器高度、列表项高度、偏移量（Transform）。
- 优化手段：设置缓冲区（Buffer）防止滚动过快白屏、处理动态高度（预估高度 + 渲染后更新）。

常见坑：

- 滚动时频繁触发计算导致卡顿（需配合 requestAnimationFrame）。
- 列表项高度不固定时的复杂逻辑处理。

追问：

- 如果列表项包含图片，高度会发生变化，怎么处理？
- 除了虚拟列表，还有哪些优化大数据量渲染的方法？

10. 谈谈 TypeScript 中 interface 和 type 的区别。

难度：★★

考点：TS 基础与进阶

答案要点：

两者都用于定义类型，但在扩展性和使用场景上有所不同。

- 扩展方式：interface 通过 extends 继承，type 通过 & 交叉类型实现。
- 声明合并：interface 支持同名自动合并，type 不支持。
- 能力差异：type 可以定义基本类型别名、联合类型、元组，interface 专注于描述对象结构。
- 建议：优先使用 interface 描述数据结构，需要复杂类型操作时用 type。

常见坑：

- 混淆了 type 的赋值和 interface 的继承。
- 忽略了 interface 无法表达联合类型的情况。

追问：

- 什么是泛型？泛型约束（extends）怎么用？
- TS 中的 keyof 和 typeof 操作符有什么作用？

11. 介绍一下你最成功或最有挑战性的一个项目。

难度：★★★

考点：综合能力与经验

答案要点：

这是一个开放性问题，重点在于体现你解决复杂问题的思路。

- 项目背景：简单描述业务场景和你的角色。
- 核心挑战：遇到了什么技术难题（如性能瓶颈、架构混乱、复杂交互）。
- 解决方案：你调研了哪些方案，为什么选择了最终的方案。
- 最终成果：量化的指标提升（如首屏时间降低 50%）或团队收益。

常见坑：

- 描述过于平淡，没有体现出“挑战性”。
- 无法清晰解释技术选型的理由。

追问：

- 如果现在让你重新做这个项目，你会做哪些改进？
- 在这个过程中你最大的收获是什么？

12. 谈谈你对前端工程化的理解。

难度：★★★

考点：工程化视野

答案要点：

工程化是为了提高开发效率、降低维护成本、保证项目质量的一系列手段。

- 规范化：代码规范（ESLint/Prettier）、提交规范（Commitlint）、目录结构规范。
- 自动化：自动化测试、CI/CD 持续集成与部署。
- 模块化/组件化：解耦业务逻辑，提高复用性。
- 工具化：脚手架（CLI）、构建工具、监控平台。

常见坑：

- 认为工程化就是写 Webpack 配置。
- 忽略了团队协作流程中的工程化实践。

追问：

- 你们团队是如何落地代码 Review 的？
- 如何设计一个通用的前端脚手架？

三面（架构与综合能力）

1. 如果让你从零设计一个前端监控系统，你会如何规划？

难度：★★★

考点：系统设计能力

答案要点：

设计监控系统需要考虑数据采集、存储、清洗和可视化展示的全链路。

- 采集端：设计 SDK，支持自动捕获（错误、性能、行为）和手动埋点，保证轻量化。
- 传输层：考虑上报频率、压缩数据、离线存储（IndexedDB）防止数据丢失。
- 后端处理：使用消息队列（Kafka）削峰填谷，实时计算引擎（Flink）处理报警。
- 展示层：看板展示核心指标、错误堆栈还原、用户行为回溯（录屏或日志流）。

常见坑：

- 忽略了监控系统本身对业务性能的影响。
- 没考虑到海量数据下的存储和查询压力。

追问：

- 如何保证埋点数据的准确性？
- 怎么实现自动化的性能异常报警？

2. 在技术选型时，你会考虑哪些因素？

难度：★★★

考点：决策与视野

答案要点：

技术选型不是选“最好的”，而是选“最合适的”。

- 业务匹配度：是否满足当前业务需求，是否有过度的技术债。
- 社区生态：文档是否完善，社区是否活跃，遇到坑是否容易找到方案。
- 团队上手成本：团队成员的技能储备，学习曲线是否平缓。
- 可维护性与扩展性：长期维护的成本，是否容易进行二次开发或迁移。

常见坑：

- 盲目追求新技术（追新族）。
- 忽略了公司的技术栈统一要求。

追问：

- 举一个你曾经参与过的技术选型案例。
- 如何说服团队成员接受一个新的技术方案？

3. 谈谈你对前端组件库设计的看法，如何保证组件的高内聚低耦合？

难度：★★★

考点：工程化与抽象能力

答案要点：

优秀的组件库应该平衡灵活性和易用性。

- 职责单一：一个组件只做一件事，复杂的组件通过组合实现。
- 属性设计：Props 命名规范，提供合理的默认值，支持受控与非受控模式。
- 样式管理：支持主题定制，避免全局样式污染（CSS Modules/CSS-in-JS）。
- 文档与测试：完善的 API 文档、交互示例以及高覆盖率的单元测试。

常见坑：

- 组件 Props 过多导致逻辑极其复杂。
- 组件间存在隐式的依赖关系。

追问：

- 如何处理组件库的按需加载？
- 怎么实现一个组件库的主题切换功能？

4. 面对复杂的业务需求，你如何进行需求拆解和任务分配？

难度：★★★

考点：项目管理与协作

答案要点：

需求拆解是将模糊的目标转化为可执行的技术方案的过程。

- 需求理解：先搞清楚业务目标和用户痛点，不盲目开始写代码。
- 模块拆解：按功能模块或页面维度拆分，识别出核心路径和边缘 Case。
- 技术方案评审：邀请团队成员参与，评估技术难点和潜在风险。
- 任务排期：预留缓冲时间（Buffer），按优先级排序，明确里程碑节点。

常见坑：

- 拆解不够细致，导致后期任务延期。
- 忽略了前后端接口联调的时间成本。

追问：

- 如果项目进度严重滞后，你会采取什么措施？
- 如何处理产品经理频繁变更需求的情况？

5. 你认为未来 2-3 年前端领域的发展趋势是什么？

难度：★★

考点：技术前瞻性

答案要点：

前端领域正从单纯的页面开发向更深层次的工程化和智能化演进。

- 边缘计算与 Serverless：前端边界进一步扩大，SSR/ISR 成为常态。
- 提效工具：Rust/Go 赋能前端工具链（如 Turbopack, Rolldown），构建速度质变。
- AI + 前端：Copilot 辅助编程、UI 自动化生成、智能监控分析。
- 跨端融合：鸿蒙生态、小程序框架的演进，追求更接近原生的体验。

常见坑：

- 回答过于笼统，没有具体的例子支撑。
- 表现出对新技术的排斥或盲目乐观。

追问：

- 你最近在关注或学习哪项新技术？
- AI 的兴起对前端开发者的职业规划有什么影响？

6. 聊聊你在团队中的角色，以及你如何帮助团队成长。

难度：★★

考点：软技能与领导力

答案要点：

团队成长是个人价值的延伸。

- 技术分享：定期进行技术分享（Tech Talk），沉淀文档和最佳实践。
- 代码质量：参与 Code Review，制定并维护团队的代码规范。
- 工具沉淀：开发内部工具或脚手架，解决团队开发中的重复劳动。
- 新人带教：指导校招或新入职同事，帮助其快速融入业务。

常见坑：

- 只关注自己的任务，对团队事务漠不关心。
- 分享内容过于理论，无法落地到实际业务。

追问：

- 如果团队成员在技术方案上产生严重分歧，你如何协调？
- 你是如何平衡业务开发和技术基建时间的？

7. 你对高途有什么了解？为什么选择加入我们？

难度：★

考点：求职动机

答案要点：

这需要候选人提前做功课，体现对公司的认可。

- 业务理解：对在线教育行业的看法，对高途主要产品线的认知。
- 技术契合：自己的技术背景如何能为高途的业务带来价值。
- 职业规划：高途的平台能为自己的职业发展提供什么样的空间。
- 态度诚恳：表达对团队氛围或技术挑战的期待。

常见坑：

- 回答过于模版化，没有针对性。
- 表现出对公司业务完全不了解。

追问：

- 你对我们公司的前端技术栈有什么建议吗？
- 你最希望在未来的工作中获得什么？